Reporte

EQUIPO 10	
Nombre: Emiliano Salas Padrón Jesus Adrian De la Cruz Dominguez Joel Guadalupe Taméz Olmos	**Matricula:** **2157333** **2156493** **2158224**
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Fundamental 3. Diseño arquitectural del sistema
Fecha: 17/09/2026	
Bibliografia (Formato APA) **Evans, E. — Domain-Driven Design (2003), Addison-Wesley** **Vernon, V. — Implementing Domain-Driven Design, Addison-Wesley** **Fowler, M. — Patterns of Enterprise Application Architecture, Addison-Wesle**	

Objetivo:

El presente documento desarrolla la actividad práctica de la Unidad 3 (diseño arquitectural del sistema) para el caso FarmaExpress, a partir del BRD de la Unidad 2. Procesa cada requerimiento de la guía de trabajo, registra los datos imprecisos o faltantes que afectan la coherencia entre el BRD y las historias de usuario, y presenta una propuesta de arquitectura pendiente de revisión del equipo.

Procedimiento:

FarmaExpress opera como una cadena farmacéutica multisucursal dedicada a la comercialización de medicamentos de venta libre, sujetos a receta médica simple y controlados, además de artículos de cuidado personal. En esta arquitectura distribuida, la complejidad del sistema no reside de forma exclusiva en el procesamiento de la transacción comercial en el punto de venta (POS), sino en la estricta alineación regulatoria y operativa requerida para garantizar la trazabilidad por lote y fecha de caducidad.

Preparación del caso

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Registrar venta en caja	Cajero (el Cliente solicita y paga)	RN-01; N-01; US01	Venta registrada y comprobante fiscal generado (N-01; ver H-15).
CU-02	Verificar receta y aplicar la política de venta según categoría; rechazar y escalar si la receta está incompleta o vencida	Cajero; Responsable de Farmacia (escalamiento)	RN-02, RN-03; N-02, N-03; US03, US04	La venta continúa según la política de su categoría, o se rechaza y se escala al Responsable de Farmacia.
CU-03	Capturar datos del comprador de un medicamento controlado	Cajero	RN-04; N-04; US05	Nombre del paciente, nombre del médico y número de matrícula profesional registrados para auditoría sanitaria (US05).
CU-04	Autorizar una excepción	Responsable de Farmacia	RN-05, RN-06; N-05; US06	Excepción autorizada o no, con evidencia de la decisión
CU-05	Cancelar una venta	Cajero (solicita, según US07); Responsable de Farmacia (autoriza)	RN-07, RN-08; N-06; US07	Venta cancelada antes del corte de caja. Después del corte queda fuera de alcance.

Patron de lógica de negocio

Patrón seleccionado: Domain Model

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Cada caso de uso es un procedimiento. Las validaciones de categoría, receta, excepción y lote se repiten en venta, excepción y cancelación.	Agrupa la lógica por tabla (producto, venta, lote). Las reglas que cruzan receta, categoría y excepción quedan repartidas.	Las reglas viven en objetos con comportamiento (venta, existencia por lote) y en una política por categoría.	N-02, N-03, N-05, N-12 (RN-02, RN-03, RN-05, RN-11)
Cambios esperados	Un cambio de parámetro obliga a revisar varios scripts.	Los cambios se localizan por tabla, pero las reglas transversales se dispersan.	Los parámetros se aíslan en objetos de valor y políticas reemplazables.	Supuestos: nivel mínimo, ventanas de 90 y 30 días y umbral de diferencias deben ser parametrizables; límites regulatorios pendientes
Interacción entre conceptos	La interacción se resuelve en el código del script; crece con cada caso.	Dificulta modelar relaciones entre varios conceptos.	Modela directamente venta ↔ receta ↔ categoría ↔ lote ↔ excepción ↔ rol.	N-02 a N-06, N-10, N-12; tabla Stakeholder
Estados e invariantes	Las invariantes se verifican manualmente en cada script.	Opera sobre conjuntos de registros; sin objeto que proteja una invariante.	Un Aggregate protege invariantes (cancelación solo antes del corte; lote retirado no vendible; autorización exclusiva).	N-05, N-06, N-10; Restricciones
Criterio adicional: lectura y reportes	Adecuado y simple para consultas puntuales.	Natural para reportes sobre conjuntos de registros.	No aporta valor en lectura; costo de modelado innecesario.	N-11, N-14, N-16 a N-19

El negocio de FarmaExpress involucra reglas complejas con dependencias cruzadas (categoría de medicamento, validez de receta, autorización exclusiva por rol, invariante de caducidad/lote retirado y ventana estricta de cancelación pre-corte de caja). Un Domain Model encapsula estas invariantes en objetos con comportamiento (Aggregates), evitando duplicar la lógica en múltiples scripts y facilitando la mantenibilidad frente a cambios regulatorios.

Aplicado al flujo de escritura del proceso principal: venta, excepción, cancelación y existencia por lote. Combinación delimitada: las consultas y reportes (N-11, N-14, N-16 a N-19) se resuelven con consultas directas de lectura, equivalentes a Transaction Script o Table Module, separadas del modelo de dominio mediante CQRS.

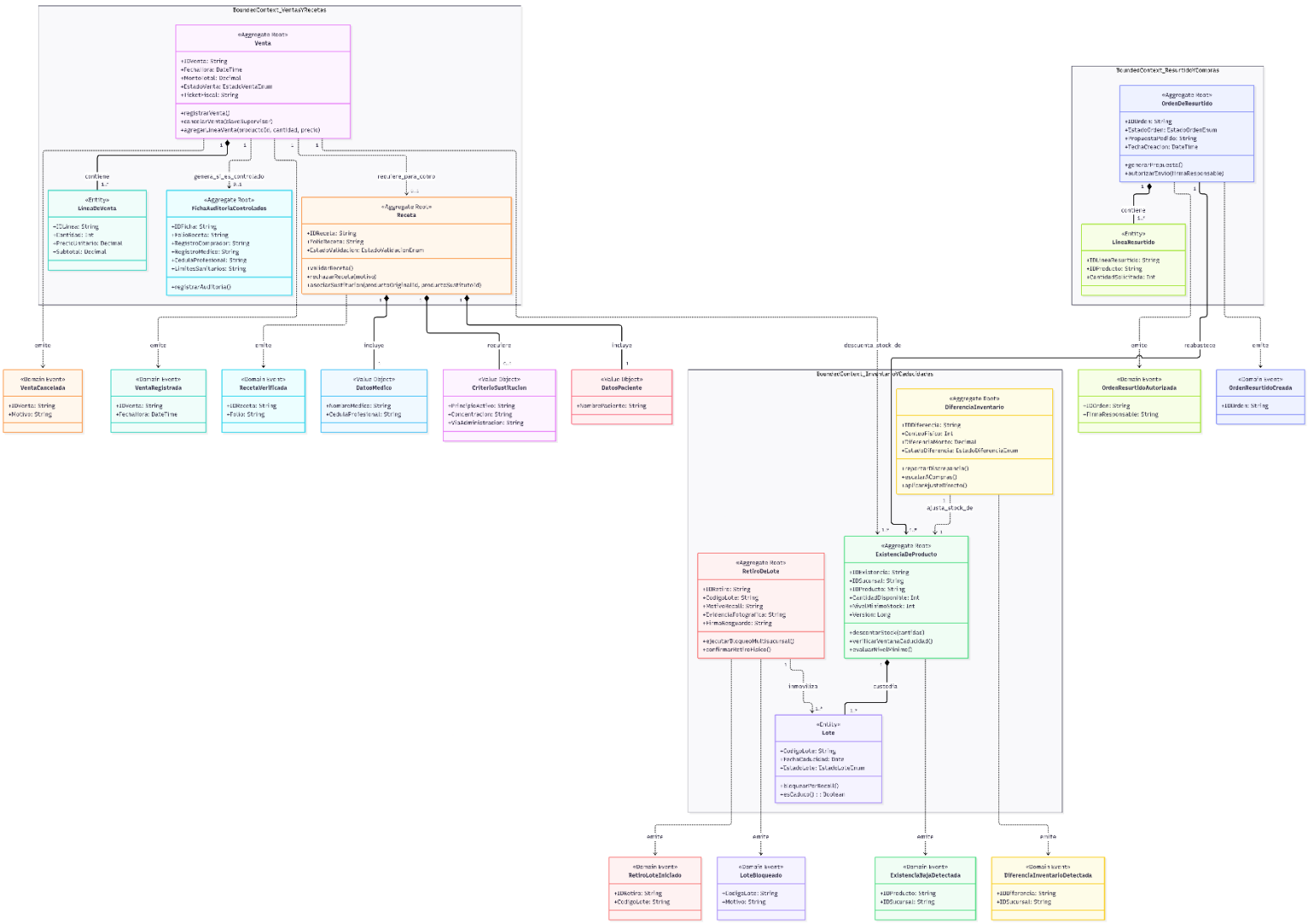
Problema concreto que ayuda a resolver es que concentra sus reglas más delicadas en pocas operaciones que combinan varios conceptos a la vez: una venta depende de la categoría del medicamento, de la receta, de una posible excepción autorizada solo por un rol y del lote del que sale el producto (N-02, N-03, N-05, N-10). Además, hay invariantes con estado: una venta solo se cancela mientras no haya corte de caja (N-06). Un Domain Model permite que esas reglas vivan junto a los datos que protegen y que no se repitan en cada caso de uso.

Segunda razón. Varios parámetros son informales o pendientes: nivel mínimo, ventanas de caducidad, umbral de diferencias y límites regulatorios (Supuestos y Restricciones). Aislarlos en objetos de valor o políticas evita editar flujos completos cuando se confirmen.

Ventajas
Reducción de costos de cambio: Al encapsular la lógica de negocio que cambia frecuentemente en un solo lugar, se minimiza el tiempo y el riesgo de errores al realizar modificaciones.
Mejora en la mantenibilidad y calidad: El código queda mejor organizado, es más legible y facilita la creación de pruebas unitarias aisladas, lo que resulta en un software más robusto y escalable a largo plazo.
Comunicación efectiva: El uso de un lenguaje ubicuo (Ubiquitous Language) alinea a los desarrolladores con los expertos del negocio, reduciendo errores de interpretación y asegurando que el software refleje fielmente las necesidades reales

Mapa de dominio

El Mapa de Dominio ilustra la estructura táctica del sistema FarmaExpress, delimitando las fronteras de los *Bounded Contexts* e identificando la jerarquía de los Aggregates y sus relaciones. La arquitectura se diseña para aislar las operaciones de alta frecuencia en el punto de venta de las tareas de gestión de inventario, resurtido centralizado y cumplimiento sanitario. Cada Aggregate actúa como una frontera de consistencia transaccional independiente. Las interacciones entre Aggregates se realizan exclusivamente mediante el intercambio de identificadores únicos de sus Raíces (*Aggregate Roots*) o a través de la emisión de Eventos de Dominio asíncronos.



Glosario de Dominio Unificado y Normalizado

ID	Término Unificado	Categoría de Dominio	Definición Precisa y Regla Asociada
1	Lote	Producto e Inventario	Conjunto de unidades de un mismo producto fabricadas en un único proceso, identificado con un código único. Es la unidad mínima de control de inventario para trazabilidad y retiro.
2	Fecha de Caducidad	Producto e Inventario	Fecha límite garantizada por el fabricante para el consumo seguro. Regla: Infranqueable; a las 00:00 horas de dicha fecha, el sistema prohíbe de forma absoluta la venta del producto.
3	Producto Próximo a Caducar	Producto e Inventario	Producto cuya Fecha de Caducidad ingresa a la ventana de alerta (90 días). Regla: Dispara alertas visuales en el sistema para asignación de salida preferente, promoción o reubicación.
4	Producto en Caducidad Crítica	Producto e Inventario	Producto cuya Fecha de Caducidad ingresa a la ventana crítica (30 días). Regla: Marca el ítem para evaluación inmediata de devolución al proveedor o retiro preventivo del anaquel.
5	Lote Retirado (Recall)	Producto e Inventario	Lote suspendido de forma inmediata por instrucción del fabricante, Sanidad o la farmacia por motivos de calidad o seguridad, independientemente de su fecha de caducidad. Exige bloqueo e inmovilización física respaldada con evidencia.
6	Existencia (Stock)	Producto e Inventario	Cantidad física disponible de un producto específico en una sucursal determinada, habilitada para la venta al público o transferencia inter-sucursal.
7	Nivel Mínimo de Existencia	Resurtido	Umbral estandarizado de inventario que, al ser traspasado a la baja, dispara el análisis de resurtido automático. Regla: Debe ser parametrizable por producto y sucursal.
8	Resurtido	Resurtido	Flujo operativo de reabastecimiento. Regla: Compras estructura la propuesta de pedido basándose en existencias bajas, pero la orden de envío al proveedor requiere la autorización del Responsable de Farmacia.
9	Medicamento de Venta Libre	Venta y Requisitos	Fármaco comercializable sin receta médica. Puede ser procesado directamente por el Cajero sin validaciones sanitarias adicionales.
10	Medicamento de Receta Simple	Venta y Requisitos	Fármaco que requiere la presentación de una receta médica vigente. Regla: El Cajero debe verificar la receta; ante inconsistencias o cambio de presentación, debe escalar al Responsable.
11	Medicamento Controlado	Venta y Requisitos	Fármaco sujeto a estricta regulación sanitaria estatal. Regla: Requiere verificación de receta, retención del documento cuando aplique, captura obligatoria de datos del comprador/médico y cumplimiento de límites de cantidad/monto.
12	Requisito Especial	Venta y Requisitos	Condición previa y obligatoria (receta, datos de cédula, autorización) que debe cumplirse en caja antes de autorizar el cobro de un producto restringido.
13	Excepción de Venta	Venta y Requisitos	Desviación del flujo normal de venta (receta con inconsistencia menor, sustitución de presentación). Regla:

			Debe ser evaluada y resuelta exclusivamente por el responsable de Farmacia.
14	Sustitución de Producto	Venta y Requisitos	Reemplazo del producto solicitado. Regla: En medicamentos con receta, el sustituto debe mantener el mismo principio activo, concentración y vía de administración. Cambios de presentación exigen autorización del Responsable.
15	Autorización de Excepción	Venta y Requisitos	Aprobación digital otorgada por el Responsable de Farmacia mediante clave/tarjeta de supervisor, que permite continuar una venta retenida por el sistema.
16	Diferencia de Inventario	Producto e Inventario	Discrepancia entre el conteo físico y el registrado en el sistema. Regla: Diferencias menores se ajustan directo; diferencias mayores requieren investigación formal e informe a Compras.
17	Conteo Físico	Producto e Inventario	Auditoría manual o cíclica ejecutada por Almacén para verificar las unidades reales en anaquel/bodega y contrastarlas contra el inventario registrado.
18	Cancelación de Venta	Venta y Requisitos	Anulación formal de una venta. Regla: Permitida únicamente el mismo día antes del corte de caja con autenticación del Responsable. Operaciones posteriores se tramitan vía Devolución/Nota de Crédito.
19	Visibilidad Multisucursal	Inventario Multisucursal	Capacidad del sistema para consultar en tiempo real el stock de productos en otras sucursales, habilitando transferencias y evitando compras redundantes por Compras.
20	Registro de Cliente	Data Objects Conectores	Data Object que almacena la información del comprador al adquirir medicamentos controlados, registrando identidad y validando los límites sanitarios por cliente.
21	Registro de Producto Caduco	Data Objects Conectores	Data Object que consolida la información detallada del producto vencido, estructurando el listado formal para su disposición final y desecho.
22	Registro de Producto Faltante	Data Objects Conectores	Data Object que almacena la información del producto agotado o insuficiente en la caja, registrando la necesidad inmediata para el pedido interno de la sucursal.
23	Registro de Producto a Reponer	Data Objects Conectores	Data Object que agrupa los productos que requieren reabastecimiento, generando la lista consolidada para el análisis del departamento central de Compras.

Fichas Técnicas Detalladas de Aggregates

La especificación técnica de los Aggregates formaliza las fronteras de consistencia transaccional y los aislamientos de estado dentro del software. Un Aggregate es un conjunto de objetos de dominio (Entidades y Value Objects) gestionados de forma unitaria respecto a los cambios de estado. Cada Aggregate posee una única Raíz (*Aggregate Root*), siendo la única entidad expuesta para referencias externas. Toda modificación en los componentes internos del Aggregate debe ejecutarse obligatoriamente a través de los métodos de la Raíz, garantizando la preservación de los invariantes de negocio.

Nombre		
Aggregate venta		
Responsabilidades	**Invariantes**	**Raiz**
Registrar la transacción comercial en el punto de venta de la sucursal. Validar la adición de productos, calculando subtotales y el monto total acumulado. Coordinar con los Aggregates de apoyo la verificación de recetas médicas y autorizaciones de excepción cuando se agregan medicamentos restringidos. Concluir el proceso de cobro y emitir el comprobante fiscal.	La venta no puede transicionar al estado *Cobrada* sin cumplir el 100% de los requisitos de receta médica para medicamentos de receta simple o controlados. La cancelación de una venta registrada solo es válida el mismo día de la operación y antes de ejecutar el corte de caja de la sucursal. La cancelación de venta exige autenticación explícita del Responsable de Farmacia mediante clave o tarjeta de supervisor.	Venta **Eventos posibles** VentaRegistrada VentaCancelada ExcepcionSolicitada

Nombre		
Aggregate Receta		
Responsabilidades	**Invariantes**	**Raiz**
Validar la vigencia e integridad de la información médica suministrada. Constatar la coincidencia técnica de principio activo, concentración y vía de administración al procesar prescripciones de medicamentos restringidos. Registrar e inmovilizar las sustituciones de producto autorizadas por el responsable de Farmacia.	Rechazar automáticamente ventas asociadas a recetas incompletas (falta de cédula, firma o paciente) o cuya fecha de prescripción haya superado el límite legal. Tras validarse y asociarse a una venta cobrada, los datos sanitarios de la receta resultan estrictamente inmutables.	Receta **Eventos posibles** RecetaValidada RecetaRechazada SustitucionRequerida

Nombre		
Aggregate FichaAuditoriaControlados		
Responsabilidades	**Invariantes**	**Raiz**
Almacenar de forma digital la trazabilidad de ventas de medicamentos controlados. Consolidar la información requerida para atender auditorías regulatorias por parte de las autoridades sanitarias (Sanidad). Evaluar el consumo acumulado por comprador y receta frente a los parámetros normativos vigentes.	Captura obligatoria de los datos del comprador, médico y cédula profesional antes de confirmar el registro. Cumplimiento de los límites de cantidad por venta y acumulados por cliente en periodos determinados de acuerdo con la regulación vigente.	FichaAuditoriaControlados **Eventos posibles** RecetaValidada RecetaRechazada SustitucionRequerida

Nombre		
Aggregate ExistenciaDeProducto		
Responsabilidades	**Invariantes**	**Raiz**
Gestionar el stock físico disponible por producto y lote en cada sucursal. Monitorear las ventanas de caducidad de los lotes custodiados (alerta 90 días, caducidad crítica 30 días). Evaluar la reducción de stock por venta o merma para identificar descensos por debajo del NivelMinimoStock.	Prohibición absoluta de venta de cualquier unidad de producto cuya FechaCaducidad haya sido alcanzada o superada. La CantidadDisponible de un producto en sucursal no puede ser menor a cero.	ExistenciaDeProducto **Eventos posibles** ExistenciaBajaDetectada ProductoEnVentanaAlerta ProductoEnCaducidadCritica

Nombre		
Aggregate ExistenciaDeProducto		
Responsabilidades	**Invariantes**	**Raiz**
Gestionar el stock físico disponible por producto y lote en cada sucursal. Monitorear las ventanas de caducidad de los lotes custodiados (alerta 90 días, caducidad crítica 30 días). Evaluar la reducción de stock por venta o merma para identificar descensos por debajo del NivelMinimoStock.	Prohibición absoluta de venta de cualquier unidad de producto cuya FechaCaducidad haya sido alcanzada o superada. La CantidadDisponible de un producto en sucursal no puede ser menor a cero.	ExistenciaDeProducto **Eventos posibles** ExistenciaBajaDetectada ProductoEnVentanaAlerta ProductoEnCaducidadCritica

Nombre		
Aggregate OrdenDeResurtido		
Responsabilidades	**Invariantes**	**Raiz**
Tramitar el flujo de reabastecimiento entre el departamento central de Compras y la sucursal. Consolidar las necesidades de reposición derivadas de las alertas de existencias bajas.	Compras elabora la propuesta de pedido basándose en el reporte de existencias bajas, pero la orden de envío al proveedor requiere obligatoriamente la autorización formal del responsable de Farmacia de la sucursal. Una vez en estado *Autorizada*, la orden de resurtido resulta inalterable en sus líneas y cantidades.	OrdenDeResurtido **Eventos posibles** OrdenResurtidoCreada OrdenResurtidoAutorizada

Nombre		
Aggregate DiferenciaInventario		
Responsabilidades	**Invariantes**	**Raiz**
Registrar y procesar las discrepancias entre el inventario físico en almacén/sucursal y el saldo en el sistema.	Discrepancias menores al umbral financiero definido se ajustan de forma directa actualizando el stock; diferencias mayores exigen investigación formal escalada con informe al Responsable de Farmacia y Compras.	DiferenciaInventario **Eventos posibles** DiferenciaInventario Detectada. DiferenciaInventario Escalada.

Nombre		
RetiroDeLote (Recall)		
Responsabilidades	**Invariantes**	**Raiz**
Ejecutar la inmovilización preventiva e inmediata de lotes dictaminados por calidad o seguridad sanitaria.	El proceso de retiro (*recall*) se activa por instrucción de seguridad o calidad, independientemente de la fecha de caducidad del lote. La emisión de la orden bloquea de forma instantánea la venta del lote en todas las sucursales. La conclusión del flujo exige adjuntar la evidencia fotográfica física (EvidenciaFotografica) del empaquetado y la firma digital de resguardo (FirmaResguardo) del Almacén.	RetiroDeLote **Eventos posibles** LoteBloqueadoMultisucursal, RetiroFisicoConfirmado.

Análisis de Decisiones Arquitectónicas (Dudosas, Pospuestas y Descartadas).

Tabla de decisiones dudosas o descartadas

ID Decisión	Categoría	Descripción del Problema / Punto Ambiguo	Estado (Pospuesta / Dudosa / Descartada)	Estrategia de Mitigación / Sustento Táctico
DEC-01	Regulación / Sanidad	Límites exactos de cantidad/monto en medicamentos controlados: Existen límites normativos por receta y cliente, pero los valores no fueron especificados en la entrevista.	Pospuesta	Derivan de regulación sanitaria externa. Se abstrae la validación en el ServicioVerificacionRegulatoria y se implementa una interfaz de integración (ACL) para parametrizar los límites formales dictados por Cumplimiento Regulatorio previo a producción.
DEC-02	Inventario / Algoritmos	Estandarización del cálculo del Nivel Mínimo de Existencia: El cálculo actual se realiza "a ojo" por cada encargado (~2 semanas para alta rotación), careciendo de documentación o fórmula oficial.	Dudosa	Se asume parametrizable por producto y sucursal en ExistenciaDeProducto. Se crea la interfaz del Domain Service ServicioCalculoMinimoStock a la espera de que el negocio defina el algoritmo matemático final basado en rotación e histórico.
DEC-03	Ventas / Reglas de Caja	Cancelaciones de venta posteriores al corte de caja: Solicitud inicial de permitir anulaciones de ventas de días anteriores directamente desde la terminal de cobro del Cajero.	Descartada	Se descarta del alcance del sistema de venta el mismo día. Las anulaciónes posteriores al corte de caja se tramitan fuera de este flujo mediante el proceso contable de Nota de Crédito / Devolución.
DEC-04	Formulación / Sustitución	Mecanismo exacto de sustitución cuando cambia la presentación: Ambigüedad sobre si el Cajero puede sustituir tabletas por cápsulas cuando se mantiene la misma concentración y vía.	Dudosa	Se establece que el cambio de presentación exige obligatoriamente la autorización explícita del Responsable de Farmacia, requiriendo coincidencia estricta en los Value Objects de PrincipioActivo, Concentracion y ViaDeAdministracion.

DEC-05	Inventario / Finanzas	Definición de umbral exacto para Diferencias Menores vs. Mayores de inventario: Falta de especificación del monto financiero en divisa que divide el ajuste directo de la investigación formal.	Pospuesta	El Aggregate DiferenciaInventario evalúa una propiedad parametrizable (MontoLimiteAjusteDirecto), cuyo valor numérico final se configurará con el stakeholder durante la fase de despliegue.
DEC-06	Inventario / Caducidad	Parametrización flexible de ventanas de caducidad (90/30 días): Las ventanas de 90 días (alerta) y 30 días (crítico) hoy son prácticas verbales que podrían diferir por categoría de producto.	Pospuesta	Se configuran como valores por defecto en el ServicioValidacionCaducidad, pero diseñadas como variables configurables por categoría de producto para admitir insumos con vidas útiles reducidas.

Service Layer

Servicio de Aplicación	Método / Caso de Uso	Colaboradores	Reglas que Delega (Lógica protegida en el Dominio)
Venta ApplicationService	RegistrarVentaMostrador	VentaRepository ExistenciaRepository PoliticaDeVentaPorCategoria *(Domain Service)* • UnitOfWork	• Validación de receta/controlados: Delegada a PoliticaDeVentaPorCategoria y al Value Object Receta (vigencia, datos de médico). • Cálculo de montos y totales: Delegado al Aggregate Root Venta y Entidad DetalleVenta. • Verificación de stock y FEFO: Delegada al Aggregate ExistenciaDeProducto y Entidad Lote (evita lotes vencidos o en recall). • Consistencia atómica: Delegada a UnitOfWork.
	AutorizarExcepcionVenta	VentaRepository	• Verificación de rol y permisos: Delegada al Aggregate Usuario (valida rol de Responsable de Farmacia).

		UsuarioRepository • UnitOfWork	• Inmutabilidad y trazabilidad de excepción: Delegada a Venta.autorizarExcepcion(), que encapsula el Value Object AutorizacionDeExcepcion e impide modificaciones posteriores al ticket.
	CancelarVentaTicket	VentaRepository CajaTurnoRepository ExistenciaRepository • UnitOfWork	• Invariante de Corte de Caja: Delegada al Aggregate CajaTurno (valida que el turno esté abierto y que la cancelación ocurra el mismo día). • Devolución de inventario: Delegada a ExistenciaDeProducto.reintegrarStock(), reactivando la disponibilidad en los lotes correspondientes.
Inventario **ApplicationService**	RegistrarRetiroPorRecall	ExistenciaRepository • UnitOfWork	• Inhabilitación de Lote: Delegada a la Entidad Lote (cambia estado a Bloqueado e inhabilita su uso en ventas de mostrador). • Disparo de evento: Delegado a ExistenciaDeProducto, que emite el Domain Event LoteBloqueado para notificar a auditoría.
Resurtido **ApplicationService**	ProponerOrdenResurtido	ExistenciaRepository OrdenResurtidoRepository • UnitOfWork	• Evaluación de umbrales: Delegada a ExistenciaDeProducto.evaluarNivelMinimo() y al Value Object NivelMinimo (compara stock disponible contra días de rotación). • Construcción de propuesta: Delegada a la fábrica/constructor del Aggregate OrdenDeResurtido.

Servicio de Aplicación	Salida (Result/DTO)	Entrada (Command/DTO)
VentaApplicationService	VentaResultadoDto *(idVenta, folioTicket, total, estado, timestamp)*	RegistrarVentaCommand *(sucursalId, cajeroId, items[{productoId, lote, cant}], recetaDto?, compradorDto?)*
	ExcepcionResultadoDto *(idVenta, estado, idExcepcion)*	AutorizarExcepcionCommand *(idVenta, idGerente, motivo, credencialesGerente)*
	ResultadoCancelacionDto *(exito, folioTicket, timestamp)*	CancelarVentaCommand *(idVenta, idCajero, motivo)*
InventarioApplicationService	ResultadoRecallDto *(numeroLote, unidadesBloqueadas)*	RegistrarRecallCommand *(numeroLote, sucursalId, motivoSanitario)*
ResurtidoApplicationService	OrdenResurtidoDto *(idOrden, totalItems, estado)*	ProponerResurtidoCommand *(sucursalId)*

Persistencia y transacciones

Repositorio	Necesidad de dominio	Métodos propuestos	Objeto que recupera o guarda
VentaRepository	Guardar una venta de mostrador, consultar tickets para cancelación del mismo día o registrar excepciones autorizadas.	AgregarAsync(Venta venta) ObtenerPorIdAsync(VentaId id) ObtenerPorFolioTicketAsync (FolioTicket folio) GuardarAsync(Venta venta)	Venta *(Aggregate Root)*[1][2] *(Incluye sus entidades DetalleVenta y Value Objects Receta , DatosComprador y AutorizacionExcepcion)*
ExistenciaRepository	Consultar inventario por sucursal, aplicar reserva de stock, descontar unidades por lote y bloquear lotes retirados sanitariamente	ObtenerPorProductoYSucursalAsync (ProductoId pId, SucursalId sId) ObtenerLotePorNumeroAsync (NumeroLote numeroLote) GuardarAsync (ExistenciaDeProducto existencia)	ExistenciaDeProducto *(Aggregate Root)*

FichaAuditoriaRepository	Registrar e inspeccionar de forma inmutable la evidencia de medicamentos controlados requerida por Sanidad	AgregarAsync (FichaAuditoriaControlados ficha) ObtenerPorFolioRecetaAsync (FolioReceta folio) ObtenerPorRangoFechaAsync (DateTime inicio, DateTime fin)	FichaAuditoriaControlados (Aggregate Root)
CajaRepository	Verificar el estado de la caja registradora del turno (Abierta/Cerrada) para validar el corte de caja antes de anular tickets	Obtener CajaActivaPorSucursalYCajeroAsync (SucursalId sId, CajeroId cId) GuardarAsync(CajaTurno caja)	CajaTurno *(Aggregate Root)*
OrdenResurtidoRepository	Registrar propuestas de reabastecimiento elaboradas por Compras y autorizadas por el responsable de Farmacia	AgregarAsync (OrdenDeResurtido orden) ObtenerPorIdAsync(OrdenId id) GuardarAsync (OrdenDeResurtido orden)	OrdenDeResurtido *(Aggregate Root)*

Unit of Works

Nombre del Unit of Work	Dónde Inicia la Transacción	Qué Cambios Incluye (Operaciones Atómicas)
RegistrarVentaMostrador	En el RegistrarVentaCommandHandler (Application Service) al invocar await _unitOfWork.BeginTransactionAsync() antes de procesar el cobro.	Inserción del encabezado en tablas de venta y detalles venta. Actualización de stock disponible y estado de la entidad Lote
AutorizarExcepcionVenta	En el AutorizarExcepcionCommandHandler tras autenticar las credenciales del Responsable de Farmacia y abrir el contexto transaccional.	Modificación del estado de la venta a "Autorizada con Excepción" en TBL_VENTA. Inserción del Value Object AutorizacionDeExcepcion con el ID del gerente, motivo y timestamp en TBL_VENTA_EXCEPCION.
CancelarVentaTicket	En el CancelarVentaCommandHandler al verificar la solicitud de anulación antes de alterar cualquier dato.	Cambio de estado del Aggregate Venta a "Cancelada" en TBL_VENTA. Reintegración de unidades al inventario (incremento de stock en TBL_LOTE y TBL_EXISTENCIA). Lectura y verificación del estado activo de CajaTurno dentro de la misma transacción.
RegistrarRecallSanitario	En el RegistrarRecallCommandHandler inmediatamente al recibir la notificación o alerta sanitaria de retiro de producto.	Actualización masiva del campo bloqueado = true en las entidades Lote afectadas en TBL_LOTE. Inserción del evento de dominio en la tabla Outbox (TBL_OUTBOX_EVENTS) para sincronización en segundo plano.
ProponerOrdenResurtido	En el ProponerResurtidoCommandHandler tras evaluar que el stock actual está por debajo del NivelMinimo.	Inserción del nuevo Aggregate Root OrdenDeResurtido en TBL_ORDEN_RESURTIDO y sus líneas en TBL_DETALLE_RESURTIDO con estado "Propuesta".

En caso de error:

Nombre del Unit of Work	Error Posible / Causante	Resultado y Acción de Mitigación (Rollback)
RegistrarVentaMostrador	Receta médica vencida, falta de datos de comprador en medicamento controlado o stock insuficiente en el lote.	RollbackAsync() Completo: Se revierte todo el proceso. No se genera el ticket de venta, no se descuenta inventario y no se crea la ficha de auditoría. La base de datos no sufre cambios.
AutorizarExcepcionVenta	El usuario que autoriza no tiene el rol de Responsable de Farmacia o el ticket pertenece a una sesión ya cerrada.	RollbackAsync() Completo: El ticket mantiene su estado original sin la excepción y se registra el evento de seguridad fallido en el log de auditoría.
CancelarVentaTicket	El turno de la caja registradora (CajaTurno) ya ejecutó el corte de caja (estado Cerrada).	RollbackAsync() Completo: Se rechaza la cancelación. No se reembolsa el dinero ni se reingresa el producto al inventario para evitar descuadres en caja.
RegistrarRecallSanitario	Caída de conexión con la base de datos durante la actualización masiva de lotes.	RollbackAsync() Completo: Ningún lote queda parcialmente bloqueado. La transacción asegura que el bloqueo de lotes por riesgo sanitario sea atómico (todos o ninguno).
ProponerOrdenResurtido	ID de sucursal inexistente o falla en el mapeo del catálogo de productos.	RollbackAsync() Completo: Se revierte el intento de generación para evitar la creación de órdenes de resurtido huérfanas o duplicadas.

Commands y Queries

En la arquitectura de FarmaExpress, la aplicación de CQRS Lógico (Command Query Responsibility Segregation) permite separar claramente las operaciones que modifican el estado del negocio (Comandos) de las operaciones que solamente consultan datos (Consultas).

Esta separación garantiza que el modelo de escritura permanezca enfocado en proteger las invariantes de negocio (usando el modelo de dominio rico con Agregados, Value Objects y el *Unit of Work*), mientras que el modelo de lectura se optimiza para devolver pantallas y reportes rápidos mediante objetos planos de transferencia de datos (DTOs / View Models) directamente desde la base de datos, sin cargar agregados ni ejecutar lógica de dominio.

Tabla de Separación CQRS (Comandos y Queries)

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Comando	Registrar Venta	IdSucursal IdCajero IdProducto NumeroLote Cantidad FolioReceta? cedulaMedico? nombrePaciente?	RegistrarVenta VentaRepository ExistenciaRepository FichaAuditoria Repository PoliticaDeVentaPor Categoria UnitOfWork	Ticket con Id venta, folio ticket, total comprador y fecha hora	Escritura rica en invariantes: Carga Agregados completos (Venta, ExistenciaDeProducto) en memoria para validar stock por lote, medicamentos vencidos y recetas sanitarias antes de aplicar cambios atómicos mediante el UnitOfWork.
Comando	Autorizar Excepción Venta	IdVenta IdGerene motivoExcepcion credenciales	AutorizarExcepcion VentaRepository UsuarioRepository UnitOfWork	Excepción, contiene id venta, id excepción y nuevo estado del ticket	**Modificación de estado y trazabilidad:** Modifica el Agregado Venta adjuntando el Value Object inmutable AutorizacionDeExcepcion. Valida el rol de Responsable de Farmacia y no realiza consultas de lectura extensas.
Query	Obtener Existencias Bajas	IdSucursal soloProximosA Caducar(bool)	Id ObtenerExistenciasBa jas	Lista de productos de bajo stock	**Lectura optimizada (Sin Agregados):** No carga el Agregado ExistenciaDeProducto ni ejecuta métodos de dominio.

Query	Obtener Reporte Auditoria	fechaInicio fechaFin IdSucursal	IdObtenerReporte Auditoria	Reporte de auditoria.	**Consulta regulatoria desnormalizada:** Cargar Agregados FichaAuditoriaControlados mediante un ORM tradicional causaría un sobrecosto inútil de memoria.

Propuesta SOLID

La propuesta de arquitectura de FarmaExpress se basa en los principios SOLID, con el objetivo de mejorar la organización del sistema, facilitar su mantenimiento y permitir que sus componentes puedan modificarse sin afectar innecesariamente a los demás.

S — Principio de responsabilidad única

Cada clase o componente debe tener una responsabilidad específica. En FarmaExpress, se propone separar las responsabilidades de ventas, recetas médicas, existencias de productos y auditoría. Los servicios de aplicación coordinarán las operaciones, mientras que los repositorios se encargarán del acceso a los datos.

Beneficio: Facilita el mantenimiento, las pruebas y la identificación de errores.

O — Principio abierto/cerrado

Los componentes deben permitir la incorporación de nuevas funcionalidades sin modificar innecesariamente el código existente. En FarmaExpress, las reglas de negocio y los umbrales de alertas podrán manejarse mediante políticas configurables. Se contemplan alertas de caducidad a 90 días y alertas críticas a 30 días.

Beneficio: Permite adaptar el sistema cuando cambien las necesidades del negocio, manteniendo controlados los cambios.

L — Principio de sustitución de Liskov

Las implementaciones de una interfaz deben respetar el comportamiento esperado por los componentes que las utilizan. En FarmaExpress, las implementaciones de los repositorios deberán cumplir los contratos establecidos y respetar las reglas del negocio, como impedir la venta de lotes caducados.

Beneficio: Permite cambiar una implementación por otra sin provocar comportamientos incorrectos en el sistema.

I — Principio de segregación de interfaces

Las interfaces deben ser específicas y contener únicamente las operaciones que necesita cada componente. En FarmaExpress, se propone separar las interfaces para ventas, existencias, auditoría y caja, evitando que un componente dependa de operaciones que no utiliza.

Beneficio: Reduce el acoplamiento y facilita las pruebas y modificaciones.

D — Principio de inversión de dependencias

Los componentes de alto nivel deben depender de abstracciones y no directamente de implementaciones concretas. En FarmaExpress, los servicios de aplicación utilizarán

interfaces para acceder a los repositorios y a la unidad de trabajo. Así, la implementación de persistencia podrá cambiarse sin modificar la lógica principal del negocio.

Beneficio: Mejora la flexibilidad, la mantenibilidad y la capacidad de evolución de la arquitectura.

Representar y documentar la arquitectura

Diagrama de clases

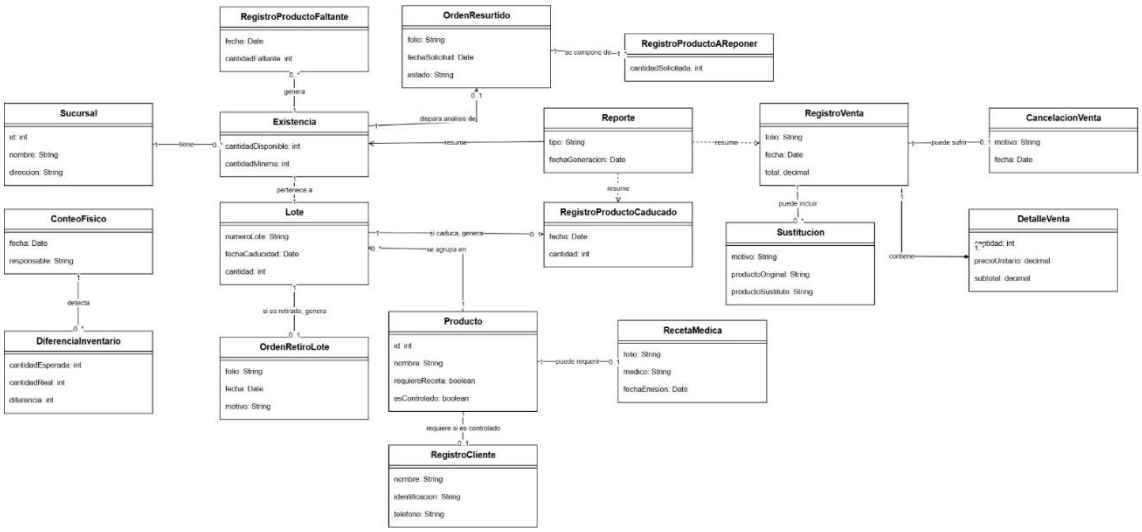
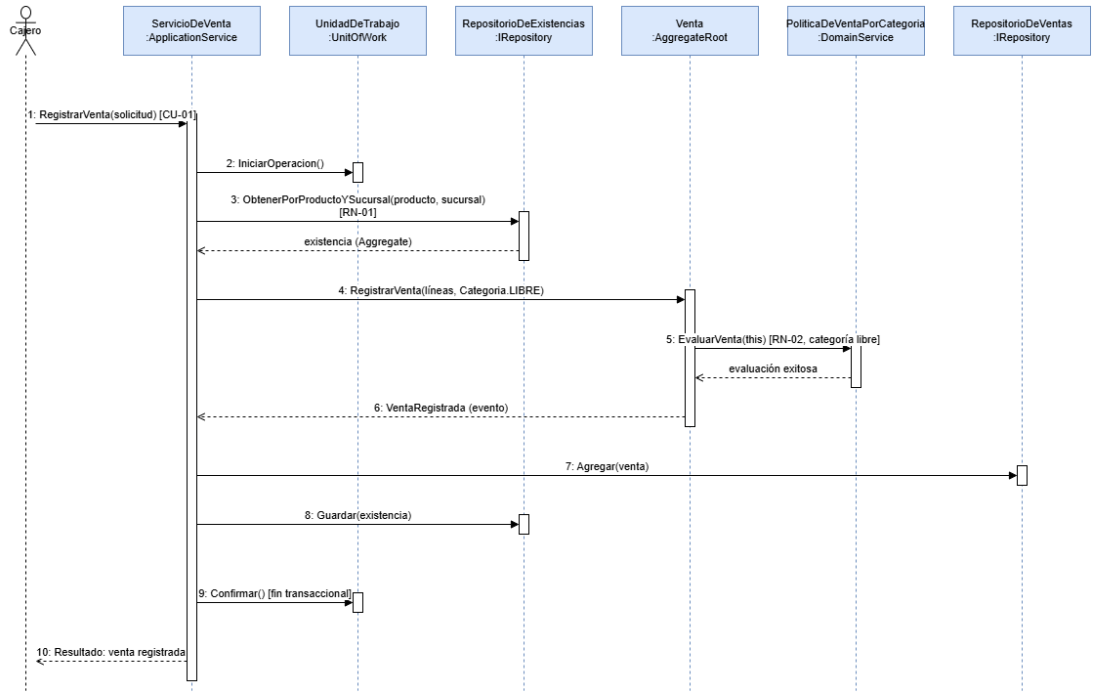


Diagrama de secuencia

Dentro del primer diagrama de secuencia tenemos los siguientes participantes:

Participante	Capa	Tipo de elemento	Que hace en el flujo
Cajero	Presentacion	Actor	Inicia la solicitud de venta en caja con los productos requeridos
ServicioDeVenta	Aplicación	Aplication Service	Coordinador de uso. Controla el inicio y cierre transaccional
UnidadDeTrabajo	Infraestructura	Unit Of Work	Garantiza la atomicidad de la transacción
RepositorioDeExistencias	Dominio	Repository Interface	Recupera y persiste el Aggregate ExistenciaDeProducto
Venta	Dominio	Aggregate Root	Entidad raíz de venta. Registra líneas y valida su propio estado
PoliticaDeVentaPorCadaCategoria	Dominio	Domain Service	Aplica reglas de negocio para la categoría de venta libre
RepositorioDeVentas	Dominio	Reposirotry	Persiste el nuevo Aggregate Venta en el almacén de datos

Diagrama de Secuencia 1: Registrar venta de medicamento libre (CU-01)

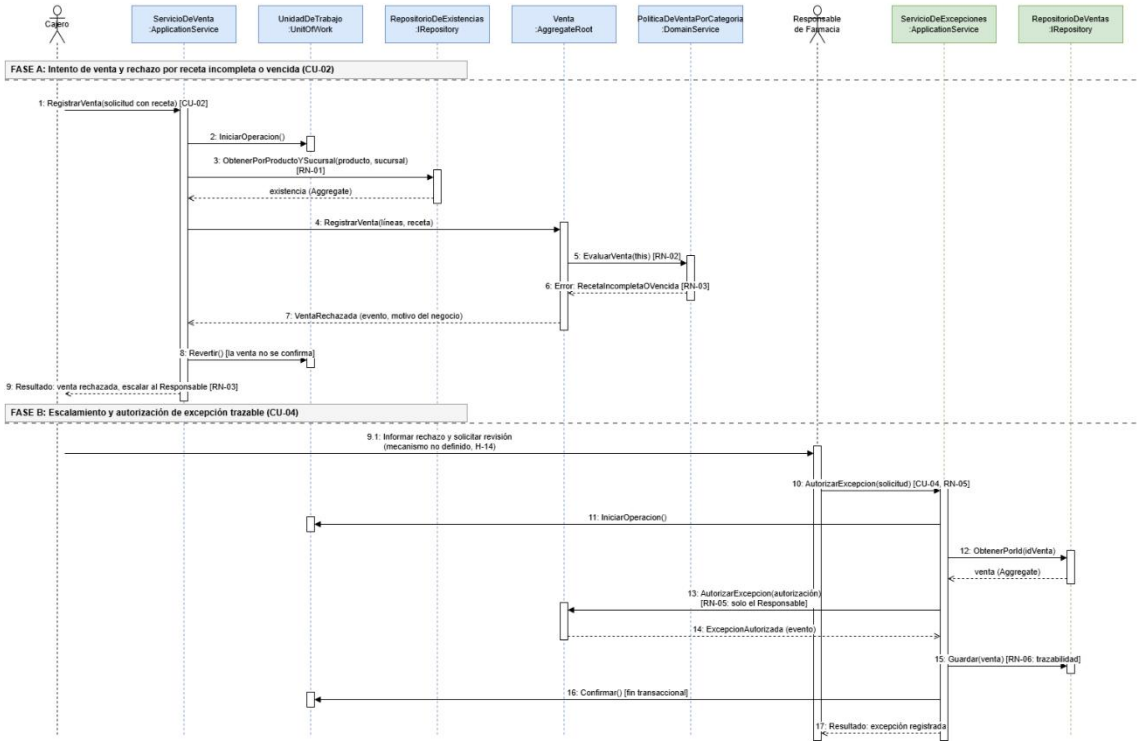


Dentro del del segundo diagrama de secuencia tenemos los siguientes participantes:

Participante	Capa	Tipo de elemento	Que hace dentro del flujo
Cajero	Presentación	Actor	Intenta registrar la venta con receta e informa del rechazo al cliente
ServicioDeVenta	Aplicación	Application Service	Coordinador del flujo de venta. Maneja el rechazo y rollback
UnidadDeTrabajo	Infraestructura	Unit of Work	Ejecuta Revertir() cuando el dominio rechaza la venta
RepositorioDeExistenciad	Dominio	Repository Interface	Consulta existencia local de medicamentos
Ventas	Dominio	Aggregate Root	Evalúa las reglas de ka receta y emite evento VentaRechazada
PolíticaDeVentaPorCategoria	Dominio	Domain Service	Detecta receta incompleta o vencida

Responsable de farmacia	Presentacion	Actor	Unico rol facultado para revisar el caso y autorizar la excepción
ServicioDeExcepciones	Aplicación	Application Service	Servicio que proceso y registra la autorizaciónDeExcepción Trazable
RepositoriosDeVentas	Dominio	Repository Interface	Guarda el registro de la excepción documentada en la venta

Diagrama de Secuencia 2: Rechazo por receta incompleta o vencida y autorización de excepción (CU-02 y CU-04)



Architecture Decisions Records

ADR-01. Domain Model como patrón de lógica de negocio

Campo del ADR	Contenido
Título	Domain Model como patrón principal para escritura; consultas directas para lectura.
Estado	Propuesta
Contexto	Reglas que combinan categoría, receta, excepción, lote y rol (N-02, N-03, N-05, N-10); invariantes con estado (N-06); parámetros modificables o pendientes (Supuestos, Restricciones).
Alternativas	Transaction Script: simple, pero repite validaciones entre venta, excepción y cancelación. Table Module: útil para reportes, pero deja dispersas las reglas que cruzan conceptos. Domain Model: protege invariantes; exige más modelado.
Decisión	Domain Model para venta, excepción, cancelación y existencia por lote; lectura con consultas directas (CQRS).
Consecuencias	Beneficios: invariantes protegidas y parámetros aislados. Costos: más modelado y mapeo con persistencia. Riesgos: las reglas pendientes (H-12 a H-20) pueden cambiar el modelo. Trabajo posterior: resolver esos hallazgos.
Trazabilidad	N-02, N-03, N-05, N-06, N-09, N-10, N-12; RN-02, RN-03, RN-05, RN-07, RN-09, RN-10, RN-11; CU-01 a CU-05.

ADR-02. Venta y ExistenciaDeProducto como Aggregates separados

Campo del ADR	Contenido
Título	Límites de consistencia entre Venta y existencia por lote.
Estado	Propuesta
Contexto	Control por producto, lote y caducidad (RN-01); inventario por producto y sucursal (Supuestos); el bloqueo de lotes cruza sucursales (N-10). El BRD no declara que la venta descuente existencia (H-16).
Alternativas	Un Aggregate único con venta e inventario: consistencia fuerte, pero de gran tamaño. Aggregates separados coordinados en una misma transacción por el servicio. Aggregates separados con consistencia eventual por eventos.
Decisión	Aggregates separados; si H-16 confirma que la venta descuenta existencia, ambos cambios se confirman en la misma UnidadDeTrabajo.
Consecuencias	Beneficios: Aggregates pequeños con raíz clara. Costos: la coordinación vive en el servicio. Riesgos: una transacción que abarque dos Aggregates exige confirmar el requisito en H-16. Trabajo posterior: decidir consistencia entre sucursales (N-14).

Trazabilidad	N-01, N-10, N-14; RN-01, RN-10; H-16.

ADR-03. Repository en términos del dominio y Unit of Work por caso de uso

Campo del ADR	Contenido
Título	Separación de persistencia y límite transaccional.
Estado	Propuesta
Contexto	La tecnología de persistencia no está definida (H-25); la guía exige que el dominio no dependa de SQL ni de detalles de infraestructura; las operaciones deben tener inicio y final comprensibles.
Alternativas	Acceso a datos directamente desde los servicios: menos código, pero acopla aplicación y dominio a la base de datos. Repository por Aggregate y UnidadDeTrabajo por método del servicio.
Decisión	Repository por Aggregate con métodos del dominio y una UnidadDeTrabajo por método del Servicio de Aplicación.
Consecuencias	Beneficios: dominio comprobable sin base de datos. Costos: interfaces e implementaciones adicionales. Riesgos: la ubicación de los datos (sucursal o central) puede obligar a ajustar la UnidadDeTrabajo. Trabajo posterior: definir tecnología.
Trazabilidad	N-01, N-05, N-06; RN-06, RN-07; H-14, H-16, H-25.

ADR-04. CQRS básico sin bases de datos separadas

Campo del ADR	Contenido
Título	Separar Commands y Queries.
Estado	Propuesta
Contexto	Las lecturas (N-11, N-14, N-16 a N-19) devuelven información entre sucursales o agregada; las escrituras protegen invariantes de un Aggregate.
Alternativas	Un único modelo para leer y escribir: menos componentes, pero obliga a exponer los Aggregates en los reportes. CQRS básico: Commands con el dominio y Queries con modelos de lectura. CQRS con bases separadas: fuera de la guía y sin requisito en el BRD.
Decisión	CQRS básico con tres Commands y tres Queries, y View Models propios de lectura.
Consecuencias	Beneficios: lecturas simples y escrituras protegidas. Costos: más clases. Riesgos: campos de los reportes aún no definidos (H-01). Trabajo posterior: completar los View Models con las US.
Trazabilidad	N-11, N-14, N-16, N-19; US02, US11, US14, US16.

Github

https://github.com/Salasdotcom/FarmaExpress

Bitacora de IA

Iteración 1				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Procesar la guía de la Unidad 3 con el BRD y señalar datos imprecisos.	Realiza una Service Layer en una tabla indicando servicio, método, entrada, colaboradores, salida y reglas que delega	Informe de arquitectura v1 en Word. Incluye hallazgos H-01 a H-27, una propuesta Domain Model + CQRS, 4 ADR, la matriz de trazabilidad y especificaciones en texto en lugar de diagramas.	Solo se contaba con el BRD. El texto de las US, el BPMN y el glosario no estaban disponibles (H-01). Los RN-xx y los nombres de componentes son propuestas, no datos del BRD..	Se le retroalimentó a la IA proporcionando más información sobre la empresa FARMA EXPRESS

Iteración 2				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Dar seguimiento al informe v1 con los demás documentos de la Unidad 2.	Seguir el documento Word ya teniendo BRD, BPMN, Glosario y US.	Contrasta BRD, US, BPMN y Glosario. Agrega los hallazgos y notas de actualización en 14 hallazgos previos.	El documento guía no venía adjunto, así que la IA hizo las referencias del informe cargado	Se le retroalimentó a la IA proporcionando más información sobre la empresa FARMA EXPRESS

Iteración 3				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Validar coherencia, corregir y revalidar	Subí el informe de arquitectura y el diagrama de clases, y pregunté si la matriz concordaba, Pedí corregirla para que fuera coherente con ambos. Si ya era coherente.	Comparación: no concordaba del todo (nombres de clases, Commands, roles y cobertura). Lista de lo que coincidía y lo que no.	La IA podrá generar respuestas ambiguas, falta de desarrollo o ideas imaginarias	Tomó los comentarios de la IA para enriquecer el planteamiento del informe, tomando lo correcto y coherente en la serie de iteraciones de validación de información

Iteración 4				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Diseñar el Mapa de Dominio rico bajo el enfoque Domain-Driven Design (DDD).	Crea un mapa de dominio de FarmaExpress con código Mermaid, glosario, fichas de aggregates y decisiones"	Generación del modelo de dominio con código Mermaid, clasificando Entidades, Value Objects, Aggregates (Venta, ExistenciaDeProducto, FichaAuditoriaControlados) y Domain Events.	Ambigüedad en la gestión de excepciones de venta.	Tomar el mapa de dominio y modificarlo para solventar la ambigüedad (se modeló AutorizacionDeExcepcion como un Value Object inmutable dentro de Venta)

Iteración 5				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Definir la Capa de Servicio (Service Layer) evitando servicios anémicos.	Realiza una Service Layer en una tabla indicando servicio, método, entrada, colaboradores, salida y reglas que delega	Tabla de orquestación con VentaApplicationService, InventarioApplicationService y ResurtidoApplicationService, delegando las reglas al dominio.	Acumular if/else y lógica de negocio en la Service Layer.	Se tomó la delegación de reglas, modificando para justificarlo a la delegación de reglas a Entidades, Value Objects y Domain Services.

Iteración 6				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Abstraer la persistencia y diseñar interfaces de Repositorio en el dominio.	Separa persistencia y transacciones con una tabla de repositorios, necesidades, métodos e implementación en infraestructura	Definición de interfaces (VentaRepository, ExistenciaRepository) usando el lenguaje ubicuo e independientes de SQL/ORM, más la propuesta de implementación en infraestructura.	Acoplar el dominio con dependencias de base de datos (DbContext/ SQL).	Se tomó la respuesta de la IA, modificándolo para aplicar el principio de Ignorancia de Persistencia (*Persistence Ignorance*).

Iteración 7				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Aplicar el patrón CQRS Lógico para separar escrituras de lecturas.	Aplica CQRS básico identificando Commands, Queries, Handlers, View Models	Especificación de RegistrarVentaCommand, AutorizarExcepcionVentaCommand, ObtenerExistenciasBajasQuery y ObtenerReporteAuditoriaControladosQuery con	Creer que CQRS exige tener dos bases de datos físicas separadas.	Se tomó la respuesta de la IA Se aclaró que basta con una segregación lógica sobre el mismo almacenamiento.

	y justificando excepciones	DTOs de lectura independientes.		

Iteración 8				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Establecer los principios de consistencia y atomicidad transaccional (Unit of Work)	ecapitula las operaciones del Unit of Work y el manejo de errores en dos tablas	Definición del ciclo de vida (BeginTransactionAsync, CommitAsync, RollbackAsync) y matriz de comportamiento ante violaciones de invariantes y fallas de red.	Inconsistencia entre el descuento de stock y la emisión del ticket.	Se tomó la respuesta de la IA, pero se corrigió la inconsistencia. Se definió un contorno atómico ACID garantizado por el UnitOfWork

Iteración 9				
Objetivo	**Prompt Resumido**	**Respuesta útil**	**Error o supuesto**	**Decisión del equipo**
Detallar el comportamiento específico del Unit of Work por cada caso de uso	Complementa indicando nombre de UoW, dónde inicia, qué cambios incluye y comportamiento ante error	Tablas detalladas para UoW_RegistrarVentaMostrador, UoW_AutorizarExcepcionVenta, UoW_CancelarVentaTicket, UoW_RegistrarRecallSanitario y UoW_ProponerOrdenResurtido.	Cancelar tickets de caja cuyo turno ya fue cerrado o aplicar bloqueos parciales de lotes.	Se tomó la respuesta de la IA. Se hizo una marcación explícita del inicio, alcance de cambios y *rollback* ante errores.

Validar la propuesta

Validación	Pregunta	Evidencia
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Las secuencias y ADR citan N, US y RN, y CU-01 a CU-05 enlazan a N y US.
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Gerente/Responsable, matrícula/cédula, paciente/comprador/cliente, nombres distintos en diagrama de clases, mapa y servicios, y español/inglés mezclados entre diagramas y servicios
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido, con justificación?	La selección de Domain Model está bien argumentada (comparativa de tres patrones, ADR-01).
Consistencia	¿Los diagramas de clase y de secuencia representan una misma arquitectura?	Los componentes reflejados entre los dos diagramas están en una misma arquitectura. Esto por el Service Layer y los Handlers
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Hay repositorios por Aggregate con interfaces y el dominio no incluye SQL.
Transacción	¿El límite de la operación y su manejo de error están definidos?	Existen tablas de UoW y de errores con rollback por caso, y las secuencias marcan inicio y fin transaccional

Conclusiones

Emiliano Salas Padrón

FarmaExpress ha sido un negocio que se sometió a muchos criterios, modelos, diagramas y casos de estudio en donde se delimita cada propiedad de este, esto para asegurar una buena calidad en todos los procesos que rodea en el negocio. Todos los procesos y decisiones que rodean este mismo negocio ha sido plasmado durante toda la escritura del documento, la investigación, identificación de patrones para elegirlo y junto la inteligencia artificial para delimitar, corregir y encontrar posibles problemas, enriquecen a gran medida el producto final del análisis del negocio, que prontamente se podrá extrapolar al mundo real, poniendo en practica en empresas o negocios reales en donde necesiten tener un control sobre todo lo que se rodea para garantizar un buen funcionamiento y una calidad dentro del negocio

Joel Guadalupe Taméz Olmos

La propuesta de arquitectura de FarmaExpress permite organizar los componentes del sistema y mejorar el control de ventas, inventario, recetas médicas y caducidad de medicamentos. La aplicación de los principios SOLID facilita el mantenimiento y la adaptación del sistema, mientras que la validación ayuda a identificar requerimientos pendientes y asegurar que la arquitectura cumpla con las necesidades del negocio. Finalmente, será necesario resolver los puntos pendientes y obtener la aprobación de los responsables para confirmar que la propuesta sea consistente, segura y funcional.

Jesús Adrián De la Cruz Dominguez

Hacer esta actividad me cambió la idea que tenía sobre construir un sistema. Pensaba que lo difícil era programar, pero en FarmaExpress lo más pesado fue entender el negocio: qué pasa cuando se vende un medicamento que requiere receta, cómo se controla lo que está por caducar o qué tan seguido hay que actualizar el inventario. Todo eso se tiene que definir antes de escribir una sola línea.

Los diagramas y la arquitectura fueron lo que más me ayudó a ordenar mis ideas. Al ponerlo todo en papel, empezaron a salir huecos y dudas que de otra forma habría descubierto muy tarde. Con SOLID pasó algo parecido: al principio parecía teoría, pero luego vi que sirve para que el sistema no sea un enredo cuando toque modificarlo.

Me quedo con que un buen sistema se nota menos en el código y más en lo bien pensado que está desde el principio. Todavía quedan puntos por resolver y falta que los responsables lo aprueben, pero siento que la base está bien armada.